Guard App — Custom Hooks

All custom hooks with full TypeScript implementation.

---

1. useGuardStatus

Manages the guard's online/offline toggle. Calls the API, updates Zustand, and starts/stops background location tracking and WebSocket connection.

```ts
// hooks/useGuardStatus.ts
import { useState } from 'react';
import { Alert } from 'react-native';
import { useMutation, useQueryClient } from '@tanstack/react-
        query';
import { useGuardStore } from '@/stores/guardStore';
import { guardProfileService } from '@/services/api/
        guardProfileService';
import { guardWebSocketService } from '@/services/websocket/
        guardWebSocketService';
import { requestLocationPermissions } from '@/lib/
        locationPermissions';
import {
  startBackgroundLocationTracking,
  stopBackgroundLocationTracking,
} from '@/lib/backgroundLocation';
import { useActiveBookingStore } from '@/stores/
        activeBookingStore';

interface UseGuardStatusReturn {
  isOnline: boolean;
  isToggling: boolean;
  toggleOnlineStatus: () => Promise<void>;
}

export function useGuardStatus(): UseGuardStatusReturn {
  const queryClient = useQueryClient();
  const { isOnline, setOnline, guard } = useGuardStore();
  const { activeBooking } = useActiveBookingStore();
  const [isToggling, setIsToggling] = useState(false);

  const goOnlineMutation = useMutation({
    mutationFn: () => guardProfileService.setOnlineStatus(true),
    onSuccess: () => {
      setOnline(true);
```

```
    // Connect WebSocket when guard goes online
    if (guard?.id) {
      guardWebSocketService.connect(guard.id);
    }
    queryClient.invalidateQueries({ queryKey: ['guard-
      profile'] });
  },
  onError: () => {
    Alert.alert('Error', 'Failed to go online. Please try
      again.');
  },
});

const goOfflineMutation = useMutation({
  mutationFn: () => guardProfileService.setOnlineStatus(false),
  onSuccess: async () => {
    setOnline(false);
    // Disconnect WebSocket
    guardWebSocketService.disconnect();
    // Stop background location if tracking (e.g. no active
      booking but somehow running)
    await stopBackgroundLocationTracking();
    queryClient.invalidateQueries({ queryKey: ['guard-
      profile'] });
  },
  onError: () => {
    Alert.alert('Error', 'Failed to go offline. Please try
      again.');
  },
});

const toggleOnlineStatus = async () => {
  if (isToggling) return;

  // Prevent going offline during an active booking
  if (isOnline && activeBooking) {
    Alert.alert(
      'Active Booking',
      'You cannot go offline while you have an active booking.
      Complete or cancel the booking first.',
      [{ text: 'OK' }]
    );
    return;
  }

  setIsToggling(true);

  try {
    if (!isOnline) {
      // Request location permissions before going online
      const hasPermission = await requestLocationPermissions();
      if (!hasPermission) {
        setIsToggling(false);
        return;
```

```
        }
        await goOnlineMutation.mutateAsync();
      } else {
        await goOfflineMutation.mutateAsync();
      }
    } finally {
      setIsToggling(false);
    }
  };

  return {
    isOnline,
    isToggling,
    toggleOnlineStatus,
  };
}
```

---

2. useIncomingBookingRequest

Subscribes to incoming booking requests via WebSocket. Plays a sound and vibrates when a new request arrives. Manages the auto-decline countdown.

```
// hooks/useIncomingBookingRequest.ts
import { useEffect, useRef } from 'react';
import { Vibration } from 'react-native';
import { Audio } from 'expo-av';
import { useActiveBookingStore } from '@/stores/
        activeBookingStore';
import { guardWebSocketService } from '@/services/websocket/
        guardWebSocketService';
import { useGuardStore } from '@/stores/guardStore';
import type { IncomingRequest } from '@/types/booking';

const AUTO_DECLINE_MS = 30_000;

export function useIncomingBookingRequest() {
  const { isOnline } = useGuardStore();
  const { setIncomingRequest, incomingRequest } =
        useActiveBookingStore();
  const soundRef = useRef<Audio.Sound | null>(null);
  const declineTimerRef = useRef<ReturnType<typeof setTimeout> |
        null>(null);

  // Preload notification sound
  useEffect(() => {
    async function loadSound() {
      try {
        const { sound } = await Audio.Sound.createAsync(
          require('@/assets/sounds/new_request.wav'),
          { shouldPlay: false, volume: 1.0 }
        );
```

```
      soundRef.current = sound;
    } catch (e) {
      console.warn('Failed to load notification sound:', e);
    }
  }

  loadSound();
  return () => {
    soundRef.current?.unloadAsync();
  };
}, []);

// Subscribe to WebSocket events
useEffect(() => {
  if (!isOnline) return;

  const handleNewRequest = async (request: IncomingRequest) => {
    // If already handling a request, ignore new ones
    if (incomingRequest) return;

    // Play sound + vibrate
    try {
      await soundRef.current?.replayAsync();
    } catch {}
    Vibration.vibrate([0, 400, 200, 400, 200, 400]);

    setIncomingRequest(request);

    // Auto-decline after countdown
    declineTimerRef.current = setTimeout(() => {
      setIncomingRequest(null);
      guardWebSocketService.send({
        type: 'booking.declined',
        payload: { booking_id: request.id, reason: 'timeout' },
      });
    }, AUTO_DECLINE_MS);
  };

  const handleRequestCancelled = (data: { booking_id: string })
      => {
    if (incomingRequest?.id === data.booking_id) {
      clearDeclineTimer();
      setIncomingRequest(null);
      Vibration.cancel();
    }
  };

  guardWebSocketService.on('booking.new_request',
      handleNewRequest);
  guardWebSocketService.on('booking.request_cancelled',
      handleRequestCancelled);

  return () => {
```

```
      guardWebSocketService.off('booking.new_request',
        handleNewRequest);
      guardWebSocketService.off('booking.request_cancelled',
        handleRequestCancelled);
    };
  }, [isOnline, incomingRequest]);

  const clearDeclineTimer = () => {
    if (declineTimerRef.current) {
      clearTimeout(declineTimerRef.current);
      declineTimerRef.current = null;
    }
  };

  // Cancel timer when request is accepted/dismissed
  useEffect(() => {
    if (!incomingRequest) {
      clearDeclineTimer();
    }
  }, [incomingRequest]);

  return { incomingRequest };
}
```

---

3. useAcceptBooking

Handles accepting a booking request with proper race condition handling. If two guards simultaneously try to accept the same booking, only one will succeed.

```
// hooks/useAcceptBooking.ts
import { useMutation, useQueryClient } from '@tanstack/react-
        query';
import { Alert } from 'react-native';
import { bookingService } from '@/services/api/bookingService';
import { useActiveBookingStore } from '@/stores/
        activeBookingStore';
import { guardWebSocketService } from '@/services/websocket/
        guardWebSocketService';
import { startBackgroundLocationTracking } from '@/lib/
        backgroundLocation';
import AsyncStorage from '@react-native-async-storage/async-
        storage';

interface UseAcceptBookingReturn {
  acceptBooking: () => Promise<void>;
  declineBooking: () => Promise<void>;
  isAccepting: boolean;
  isDeclining: boolean;
}
```

```
export function useAcceptBooking(bookingId: string):
        UseAcceptBookingReturn {
  const queryClient = useQueryClient();
  const { setActiveBooking, setIncomingRequest } =
        useActiveBookingStore();

  const acceptMutation = useMutation({
    mutationFn: () => bookingService.acceptBooking(bookingId),
    onSuccess: async (booking) => {
      setIncomingRequest(null);
      setActiveBooking(booking);

      // Persist token for background task
      const token = await AsyncStorage.getItem('auth_token');
      // startBackgroundLocationTracking reads token from
        AsyncStorage directly

      // Start background location tracking
      const result = await
        startBackgroundLocationTracking(booking.id);
      if (!result.success) {
        console.warn('Background location failed to start:',
        result.error);
      }

      // Notify backend via WebSocket
      guardWebSocketService.send({
        type: 'booking.accepted',
        payload: { booking_id: booking.id },
      });

      queryClient.invalidateQueries({ queryKey: ['active-
        booking'] });
    },
    onError: (error: any) => {
      if (error?.response?.status === 409) {
        // Race condition — booking was already accepted by
        another guard
        setIncomingRequest(null);
        Alert.alert(
          'Booking Taken',
          'This booking was already accepted by another guard.',
          [{ text: 'OK' }]
        );
      } else {
        Alert.alert('Error',
        'Failed to accept booking. Please try again.');
      }
    },
  });

  const declineMutation = useMutation({
    mutationFn: () => bookingService.declineBooking(bookingId),
    onSuccess: () => {
```

```
      setIncomingRequest(null);
      guardWebSocketService.send({
        type: 'booking.declined',
        payload: { booking_id: bookingId, reason:
        'guard_declined' },
      });
    },
  });

  return {
    acceptBooking: () => acceptMutation.mutateAsync(),
    declineBooking: () => declineMutation.mutateAsync(),
    isAccepting: acceptMutation.isPending,
    isDeclining: declineMutation.isPending,
  };
}
```

---

4. useActiveBooking

Provides the current active booking from Zustand store and subscribes to WebSocket updates to keep it in sync.

```
// hooks/useActiveBooking.ts
import { useEffect } from 'react';
import { useQuery } from '@tanstack/react-query';
import { useActiveBookingStore } from '@/stores/
        activeBookingStore';
import { bookingService } from '@/services/api/bookingService';
import { guardWebSocketService } from '@/services/websocket/
        guardWebSocketService';
import { stopBackgroundLocationTracking } from '@/lib/
        backgroundLocation';
import type { ActiveBooking } from '@/types/booking';

export function useActiveBooking() {
  const { activeBooking, setActiveBooking, updateBookingStatus } =
    useActiveBookingStore();

  // Fetch from API on mount (restores state after app restart)
  const { data: serverBooking } = useQuery({
    queryKey: ['active-booking'],
    queryFn: bookingService.getActiveBooking,
    retry: false,
  });

  useEffect(() => {
    if (serverBooking && !activeBooking) {
      setActiveBooking(serverBooking);
    }
  }, [serverBooking]);
```

```
  // Subscribe to WebSocket status changes
  useEffect(() => {
    const handleStatusChanged = async (data: {
      booking_id: string;
      status: ActiveBooking['status'];
    }) => {
      if (data.booking_id !== activeBooking?.id) return;

      updateBookingStatus(data.status);

      if (data.status === 'completed' || data.status ===
        'cancelled') {
        await stopBackgroundLocationTracking();
        setTimeout(() => setActiveBooking(null), 3000);
      }
    };

    guardWebSocketService.on('booking.status_changed',
        handleStatusChanged);
    return () =>
        guardWebSocketService.off('booking.status_changed',
        handleStatusChanged);
  }, [activeBooking?.id]);

  return {
    activeBooking: activeBooking ?? serverBooking ?? null,
    hasActiveBooking: !!(activeBooking ?? serverBooking),
  };
}
```

---

5. useNavigationToUser

Opens the device's native navigation app (Google Maps on Android, Apple Maps / Google Maps on iOS) with directions to the user's pickup location.

```
// hooks/useNavigationToUser.ts
import { useCallback } from 'react';
import { Platform, Alert, Linking } from 'react-native';
import type { ActiveBooking } from '@/types/booking';

interface UseNavigationToUserReturn {
  navigateToUser: () => void;
  openInGoogleMaps: () => void;
  openInAppleMaps: () => void;
}

export function useNavigationToUser(
  booking: ActiveBooking | null
): UseNavigationToUserReturn {
  const lat = booking?.pickup_location.lat;
```

```
  const lon = booking?.pickup_location.lon;
  const label = encodeURIComponent(booking?.user_name ?? 'Pickup
        Location');

  const openInGoogleMaps = useCallback(() => {
    if (!lat || !lon) return;
    const url = `google.navigation:q=${lat},${lon}&mode=d`;
    const fallbackUrl = `https://www.google.com/maps/dir/?
        api=1&destination=${lat},${lon}&travelmode=driving`;

    Linking.canOpenURL(url)
      .then((supported) => Linking.openURL(supported ? url :
        fallbackUrl))
      .catch(() => Linking.openURL(fallbackUrl));
  }, [lat, lon]);

  const openInAppleMaps = useCallback(() => {
    if (!lat || !lon) return;
    const url = `maps://app?daddr=${lat},${lon}&dirflg=d`;
    Linking.openURL(url).catch(() =>
      Alert.alert('Error', 'Could not open Maps.')
    );
  }, [lat, lon]);

  const navigateToUser = useCallback(() => {
    if (!lat || !lon) {
      Alert.alert('Error', 'Pickup location not available.');
      return;
    }

    if (Platform.OS === 'android') {
      // Android: always open Google Maps
      openInGoogleMaps();
    } else {
      // iOS: let user choose
      Alert.alert('Navigate', 'Open directions in:', [
        { text: 'Google Maps', onPress: openInGoogleMaps },
        { text: 'Apple Maps', onPress: openInAppleMaps },
        { text: 'Cancel', style: 'cancel' },
      ]);
    }
  }, [lat, lon, Platform.OS]);

  return { navigateToUser, openInGoogleMaps, openInAppleMaps };
}
```

---

6. useEarnings

Fetches earnings summary and weekly chart data.

```ts
// hooks/useEarnings.ts
import { useQuery } from '@tanstack/react-query';
import { earningsService } from '@/services/api/earningsService';
import { useEarningsStore } from '@/stores/earningsStore';
import { useEffect } from 'react';

export function useEarnings() {
  const { setSummary } = useEarningsStore();

  const summaryQuery = useQuery({
    queryKey: ['earnings-summary'],
    queryFn: earningsService.getSummary,
    staleTime: 1000 * 60 * 5, // 5 minutes
  });

  const payoutHistoryQuery = useQuery({
    queryKey: ['payout-history'],
    queryFn: earningsService.getPayoutHistory,
    staleTime: 1000 * 60 * 10, // 10 minutes
  });

  useEffect(() => {
    if (summaryQuery.data) {
      setSummary(summaryQuery.data);
    }
  }, [summaryQuery.data]);

  return {
    summary: summaryQuery.data,
    payoutHistory: payoutHistoryQuery.data ?? [],
    isLoadingSummary: summaryQuery.isLoading,
    isLoadingPayouts: payoutHistoryQuery.isLoading,
    refetch: () => {
      summaryQuery.refetch();
      payoutHistoryQuery.refetch();
    },
  };
}
```

---

7. useDocumentUpload

Manages the full document upload flow: picks an image, gets a pre-signed S3 URL from the backend, uploads directly to S3, then confirms the upload to the backend.

```ts
// hooks/useDocumentUpload.ts
import { useState, useCallback } from 'react';
import { documentService } from '@/services/api/documentService';

interface UploadDocumentParams {
```

```
  documentType: string;
  uri: string;
  mimeType: string;
  fileName: string;
}

interface UploadProgress {
  documentType: string;
  progress: number; // 0–1
  status: 'idle' | 'uploading' | 'success' | 'error';
  error?: string;
}

interface UseDocumentUploadReturn {
  uploadDocument: (params: UploadDocumentParams) =>
        Promise<string>;
  uploadProgress: Record<string, UploadProgress>;
  isUploading: boolean;
}

export function useDocumentUpload(): UseDocumentUploadReturn {
  const [uploadProgress, setUploadProgress] = useState<
    Record<string, UploadProgress>
  >({});

  const updateProgress = (
    documentType: string,
    update: Partial<UploadProgress>
  ) => {
    setUploadProgress((prev) => ({
      ...prev,
      [documentType]: {
        documentType,
        progress: 0,
        status: 'idle',
        ...prev[documentType],
        ...update,
      },
    }));
  };

  const uploadDocument = useCallback(
    async ({
      documentType,
      uri,
      mimeType,
      fileName,
    }: UploadDocumentParams): Promise<string> => {
      updateProgress(documentType, { status: 'uploading',
        progress: 0 });

      try {
        // Step 1: Get pre-signed S3 URL from backend
```

```
      const { upload_url, s3_key } =
        await documentService.getUploadURL({
          document_type: documentType,
          file_name: fileName,
          content_type: mimeType,
        });

      updateProgress(documentType, { progress: 0.2 });

      // Step 2: Read file as blob
      const response = await fetch(uri);
      const blob = await response.blob();

      updateProgress(documentType, { progress: 0.4 });

      // Step 3: Upload directly to S3 via pre-signed URL
      const uploadResponse = await fetch(upload_url, {
        method: 'PUT',
        headers: {
          'Content-Type': mimeType,
        },
        body: blob,
      });

      if (!uploadResponse.ok) {
        throw new Error(`S3 upload failed: $
      {uploadResponse.status}`);
      }

      updateProgress(documentType, { progress: 0.8 });

      // Step 4: Confirm upload with backend (save s3_key)
      await documentService.confirmUpload({
        document_type: documentType,
        s3_key,
      });

      updateProgress(documentType, { progress: 1, status:
      'success' });

      return s3_key;
    } catch (error: any) {
      updateProgress(documentType, {
        status: 'error',
        error: error.message,
      });
      throw error;
    }
  },
  []
);

const isUploading = Object.values(uploadProgress).some(
```

```
    (p) => p.status === 'uploading'
  );

  return { uploadDocument, uploadProgress, isUploading };
}
```

---

8. useAvailabilitySchedule

Fetches and updates the guard's weekly availability schedule.

```
// hooks/useAvailabilitySchedule.ts
import { useMutation, useQuery, useQueryClient } from '@tanstack/
        react-query';
import { Alert } from 'react-native';
import { guardProfileService } from '@/services/api/
        guardProfileService';
import type { AvailabilitySchedule } from '@/types/guard';

export function useAvailabilitySchedule() {
  const queryClient = useQueryClient();

  const { data: schedule, isLoading } = useQuery({
    queryKey: ['availability-schedule'],
    queryFn: guardProfileService.getAvailabilitySchedule,
  });

  const updateSchedule = useMutation({
    mutationFn: (newSchedule: AvailabilitySchedule) =>
      guardProfileService.updateAvailability(newSchedule),
    onSuccess: (updatedSchedule) => {
      queryClient.setQueryData(['availability-schedule'],
        updatedSchedule);
    },
    onError: () => {
      Alert.alert('Error', 'Failed to update availability. Please
        try again.');
    },
  });

  const toggleDay = (day: keyof AvailabilitySchedule) => {
    if (!schedule) return;
    const updated: AvailabilitySchedule = {
      ...schedule,
      [day]: {
        ...schedule[day],
        enabled: !schedule[day].enabled,
      },
    };
    updateSchedule.mutate(updated);
  };
```

```
  const updateDayHours = (
    day: keyof AvailabilitySchedule,
    startTime: string,
    endTime: string
  ) => {
    if (!schedule) return;
    const updated: AvailabilitySchedule = {
      ...schedule,
      [day]: {
        ...schedule[day],
        start_time: startTime,
        end_time: endTime,
      },
    };
    updateSchedule.mutate(updated);
  };

  return {
    schedule,
    isLoading,
    isSaving: updateSchedule.isPending,
    toggleDay,
    updateDayHours,
  };
}
```

---

9. useBookingTimer

Counts up elapsed time for an active booking that has been started. Returns a formatted string like `"01:23:45"`.

```
// hooks/useBookingTimer.ts
import { useState, useEffect, useRef } from 'react';

function formatDuration(seconds: number): string {
  const h = Math.floor(seconds / 3600);
  const m = Math.floor((seconds % 3600) / 60);
  const s = seconds % 60;
  return [h, m, s].map((v) => String(v).padStart(2,
        '0')).join(':');
}

export function useBookingTimer(startedAt?: string): string {
  const [elapsed, setElapsed] = useState(0);
  const intervalRef = useRef<ReturnType<typeof setInterval> |
        null>(null);

  useEffect(() => {
    if (!startedAt) {
      setElapsed(0);
```

```
      if (intervalRef.current) clearInterval(intervalRef.current);
      return;
    }

    const startTime = new Date(startedAt).getTime();

    // Calculate initial elapsed time (accounts for timer
        continuing after app restart)
    const initialElapsed = Math.floor((Date.now() - startTime) /
        1000);
    setElapsed(Math.max(0, initialElapsed));

    intervalRef.current = setInterval(() => {
      const now = Date.now();
      setElapsed(Math.floor((now - startTime) / 1000));
    }, 1000);

    return () => {
      if (intervalRef.current) clearInterval(intervalRef.current);
    };
  }, [startedAt]);

  return formatDuration(elapsed);
}
```